

Kế hoạch viết tài liệu Transformer — LAB Week 1

Mục tiêu:

Tài liệu hoàn chỉnh về Transformer:

bối cảnh → cơ chế → chi tiết kiến trúc → tác động.

Cấu trúc tổng thể (13 slots)

PHẦN 1 — NHÓM BỐI CẢNH (2 người)

- **Bài toán Sequence Transduction và hạn chế của RNN/LSTM:** Trần Nguyên

- Seq2Seq là gì? Encoder nén toàn bộ input vào fixed context vector
- Vấn đề: sequential processing (không parallel), vanishing gradient, $O(n)$ path length
- Long-term dependencies bị degraded sau ~20 từ
- Người viết: Trần Nguyên

- **Attention trước Transformer: Bahdanau (2014):** Nhật Minh

- Ý tưởng: dynamic alignment — mỗi bước decoder "nhìn" vào source khác nhau
- Additive attention: $e_{ij} = v^T \tanh(W[s_{i-1}; h_j])$
- Giải quyết bottleneck của fixed vector, performance ổn định với mọi độ dài câu
- Nhưng vẫn sequential (BiRNN), chưa parallelizable
- Người viết: Nhật Minh

PHẦN 2 — KIẾN TRÚC TRANSFORMER (6 người)

- **Tổng quan Encoder-Decoder Architecture:** HLong

- Block diagram tổng thể: N=6 layers encoder + N=6 layers decoder
- Mỗi layer: Multi-Head Attention → Add&Norm → FFN → Add&Norm => tác dụng mỗi layer
- $d_{\text{model}}=512$, $h=8$, $d_k=d_v=64$, $d_{\text{ff}}=2048$
- Data flow từ input embedding → encoder stack → decoder stack → linear → softmax
- người viết : HLong

- **Scaled Dot-Product Attention:** Trâm

- Q (query): vị trí hiện tại đang "tìm gì"
- K (key): mỗi vị trí có "gì"
- V (value): nội dung thực tế để tổng hợp
- Công thức: **Attention(Q,K,V) = softmax(QK^T/√d_k)V**
- Tại sao scale bởi √d_k? — giữ gradient ổn định khi d_k lớn
- So sánh additive vs dot-product vs scaled dot-product
- Người viết: Trâm
- **Multi-Head Attention: Nhật Anh**
 - Tại sao cần nhiều head? —> học nhiều khía cạnh quan hệ khác nhau
 - Cơ chế: **QW_i^Q, KW_i^K, VW_i^V → attention → concat → W^O**
 - chia head = 8, mỗi head d_k=d_v=64 → tổng d_{model}=512
 - Trực quan: mỗi head học syntactic (dependency) vs semantic (ngữ nghĩa) khác nhau
 - Người viết: Nhật Anh
- **Positional Encoding: Quang**
 - Vì sao self-attention cần thông tin vị trí? (permutation invariant)
 - Công thức sinusoid: **PE(pos,2i)=sin(pos/10000^(2i/d_{model}))**
 - Tại sao sinusoid? — tại sao không là (+ index/ chuẩn hóa [0;1])
 - Cộng (không concat) vào input embedding, embedding scaled bởi √d_{model}
 - Alternatives: RoPE (LLaMA), ALiBi, **learned positional embeddings** (BERT/GPT)
 - Người viết: Quang
- **Encoder Layer chi tiết (IMPORTANT): Hà Anh**
 - Sub-layer 1: Multi-Head Self-Attention (Q=K=V đến từ encoder)
 - Sub-layer 2: Position-wise FFN (2 linear layers + ReLU)
 - Residual connection + LayerNorm sau mỗi sub-layer
 - Walkthrough: input "I love AI" → self-attention weights → FFN → output
 - Người viết: Hà Anh
- **Decoder Layer chi tiết (IMPORTANT): Đức Dũng**
 - Khác encoder ở 2 điểm: Masked Self-Attention + Cross-Attention
 - Masked: không cho nhìn future tokens (causal/autoregressive)
 - Cross-Attention: Q từ decoder, K,V từ encoder output
 - Sinh từng token một lúc inference, parallel khi training
 - Người viết: Đức Dũng

PHẦN 3 — Training (2 người)

- **Training: Adam, Learning Rate Schedule, Regularization:** Trung Tín, Đức

- Adam: $\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=1e-9$ (2/4)
- LR schedule: warmup 4000 steps $\rightarrow$ decay proportional to $1/\sqrt{(\text{step_num})}$
- Dropout $P_{\text{drop}}=0.1$ (trên attention weights, FFN, embeddings)
- Label Smoothing $\epsilon_{\text{ls}}=0.1$ (giảm overconfidence, cải thiện BLEU)
- Shared embeddings (encoder/decoder/pre-softmax) scale bởi $\sqrt{d_{\text{model}}}$
- Người thực hiện: Trung Tín, Đức

- **Kết quả thực nghiệm & Ablation Studies:** Hoàng Minh

- WMT 2014 EN-DE: 28.4 BLEU (+2.0 over SOTA)
- WMT 2014 EN-FR: 41.8 BLEU s đã(SOTA, single model)
- Training cost: 12h (base), 3.5 days (big) trên 8 P100 GPUs
- Ablation: giảm h (heads) $\rightarrow$ BLEU giảm, tăng d_k $\rightarrow$ không cải thiện
- So sánh trực tiếp với Seq2Seq + attention: quality cao hơn, train nhanh hơn 10x
- Người thực hiện: Hoàng Minh

PHẦN 4 — PHÂN TÍCH & TÁC ĐỘNG (2 người)

- **Ưu điểm/ Nhược điểm của Transformer:** An Phú

- Parallelization: không sequential dependency $\rightarrow$ train nhanh hơn RNN 10-100x
- $O(1)$ path length: bất kỳ 2 vị trí nào cũng kết nối trực tiếp
- Multi-head attention: học nhiều kiểu quan hệ cùng lúc
- Attention visualization: interpretable alignment patterns
- Scalability: hoạt động tốt với cả model nhỏ và rất lớn
- Người viết: An Phú

- **Từ Transformer đến LLM hiện đại:** Đức Dũng

- Decoder-only: GPT (autoregressive language modeling)
- Encoder-only: BERT (masked language modeling)
- Encoder-decoder: T5, mT5 (text-to-text framework)
- Modern innovations: RoPE (LLaMA), Grouped Query Attention, Flash Attention, MoE
- Scaling laws: từ 65M parameters (Transformer base) đến 1T+ parameters
- Người viết: Đức Dũng

WORKFLOW

- **File chung:** Một file gg docs, mỗi người viết vào section của mình
- **Comment:** Đặt câu hỏi ngay trong file dạng `Question: ...`
- **Yêu cầu tối thiểu:** mỗi section có ít nhất 1 diagram (ASCII hoặc image), 1 formula (nếu relevant), và 1 example cụ thể/ trực quan (gonna talk bout this most of the time i guess)

REFERENCE (that I found)

- **Positional Encoding:**
 Why Transformers Need Positional Encoding | Sin & Cos Explained ...
- **Transformer Architecture:**  Transformer Architecture Explained
- **Attention (FULL):**
 Attention in transformers, step-by-step | Deep Learning Chapter 6
- **Multi head attention:** <https://arxiv.org/pdf/1905.09418> (what a head learnt)
-  Let's build GPT: from scratch, in code, spelled out.
- 3D interactive walkthrough của GPT-style transformer:
<https://bbycroft.net/llm>
- GPT-2 small ngay trong trình duyệt (ONNX Runtime):
<https://poloclub.github.io/transformer-explainer>
- to be continued.

FORMAT

[Tên section] [Người viết]

1. Mục tiêu

Phần này giúp người đọc hiểu điều gì?

2. Vấn đề / Bối cảnh

Trước phần này, mô hình đang gặp vấn đề gì?

3. Ý tưởng chính

Giải thích bằng ngôn ngữ đơn giản, chưa cần công thức.

4. Cơ chế hoạt động

Giải thích từng bước: input → xử lý → output.

5. Công thức / Thuật toán

Ghi công thức chính nếu có, rồi giải thích từng ký hiệu.

6. Ví dụ cụ thể

Dùng một ví dụ ngắn để minh họa phần này hoạt động như thế nào.

7. Diagram / Minh họa

Thêm ít nhất 1 hình, ASCII diagram (don't) , bảng, hoặc flowchart, hoặc làm video dùng Manim.

8. Vì sao quan trọng?

Nếu bỏ phần này đi thì Transformer gặp vấn đề gì?

9. Dễ nhầm ở đâu?

Liệt kê 3–5 hiểu nhầm phổ biến.

10. Kết nối với phần khác

Phần này nối với section trước/sau như thế nào?

11. Tóm tắt

3–5 bullet tổng kết.

12. Tham khảo

Ghi nguồn đã dùng.

***Lưu ý:**

- Không cần follow format hoàn toàn, căn cứ vào topic => thêm/ bớt section phù hợp
- Đặt câu hỏi xuống dưới từng section nếu có thắc mắc

====SUBJECT TO CHANGE====

[BỐI CẢNH - Sequence Transduction & RNN/LSTM] [Trần Nguyên]

Mục tiêu

- Bản chất của bài toán Sequence Transduction
- Cơ chế hoạt động cơ bản của Seq2Seq
- Nguyên nhân các kiến trúc Recurrent Neural Networks (RNN/LSTM) truyền thống lại gặp rào cản lớn về tốc độ và khả năng lưu giữ thông tin dài hạn.

Bối cảnh

- Trước đây, Deep Neural Networks(mạng nơ-ron học sâu) rất mạnh mẽ trong việc xử lý các vấn đề khó khăn như nhận dạng đối tượng trực quan. Tuy nhiên, khi áp dụng DNN cho bài toán Sequence Transduction - bài toán ánh xạ 1 chuỗi đầu vào sang một chuỗi đầu ra, điển hình là dịch máy hay nhận dạng giọng nói, DNN lại tỏ ra kém hiệu quả. Lí do là vì DNN bị giới hạn bởi việc đầu vào và ra phải có số chiều nhất định, mà các bài toán như dịch máy lại là các bài toán chuỗi có độ dài đầu vào và đầu ra không xác định trước, đồng thời thứ tự giữa các từ lại rất phức tạp.
- Để giải quyết vấn đề này, kiến trúc Sequence to sequence(Seq2Seq) ra đời.

Ý tưởng chính

- Kiến trúc Seq2Seq xử lý vấn đề chuỗi có độ dài linh hoạt bằng cách chia mô hình thành 2 khối chính. Khối đầu tiên, sử dụng 1 mạng RNN/LSTM đóng vai trò là Encoder để đọc toàn bộ chuỗi văn bản đầu vào từng bước một, sau đó nén toàn bộ ý nghĩa của chuỗi đó thành 1 vector duy nhất có kích thước cố định. Tiếp theo, khối thứ hai, một mạng RNN khác đóng vai trò là Decoder sẽ sử dụng chính vector cố định này để giải mã và sinh ra chuỗi đầu ra với độ dài bất kỳ.
- Bản chất của RNN/LSTM là duyệt qua dữ liệu theo từng bước thời gian, tức là tại bước t , trạng thái ẩn h_t được tính dựa trên đầu vào hiện tại và trạng thái ẩn trước đó h_{t-1} .

Hạn chế:

- Vấn đề Sequential Processing: Vì h_t bắt buộc phải chờ h_{t-1} tính xong, mô hình phải tính toán tuần tự từ trái qua phải. Điều này sẽ khiến cho tốc độ train bị kéo dài đáng kể, gây ra giới hạn về bộ nhớ khi phải xử lí 1 vecto quá dài.
- Để mô hình học được mối liên hệ giữa từ đầu tiên và từ cuối cùng trong câu, thông tin phải truyền qua $O(n)$ bước. Vector đầu vào càng dài, quá trình lan truyền ngược càng dễ dẫn đến hiện tượng vanishing gradient, khiến mô hình thất bại trong việc học logic giữa các từ nằm cách xa nhau trong câu.
- Việc Encoder bắt buộc phải nén toàn bộ thông tin của một câu dù ngắn hay dài vào đúng một vector có số chiều cố định tạo ra một bottleneck về bộ nhớ. Càng về sau,

thông tin của những từ ở đầu câu càng dễ bị mờ nhạt hoặc biến mất do mô hình phải học thêm thông tin của những từ cuối câu vào cùng một vector. Thực tế, long-term dependencies thường bị suy giảm nghiêm trọng sau khoảng 20 từ.

[Attention trước Transformer: Bahdanau (2014)] [Nhật Minh]

1. Mục tiêu

Phần này giúp người đọc hiểu:

- Vì sao mô hình Seq2Seq ban đầu hoạt động chưa tốt với câu dài
- Attention ra đời để giải quyết vấn đề gì
- Cách mô hình tập trung vào các từ quan trọng khi dịch

2. Vấn đề / Bối cảnh

Mô hình Seq2Seq ban đầu gồm 2 khối chính là encoder và decoder. encoder có vai trò đọc toàn bộ chuỗi văn bản đầu vào từng bước một, sau đó nén toàn bộ ý nghĩa của chuỗi đó thành 1 vector ngữ cảnh duy nhất có kích thước cố định. Sau đó decoder sẽ sử dụng chính vector ngữ cảnh này để sinh ra chuỗi đầu ra với độ dài bất kỳ

Vấn đề gặp phải: Khi input quá dài, việc nén toàn bộ thông tin trong input vào 1 vector ngữ cảnh với kích thước cố định gây ra sự mất mát thông tin. Câu càng dài thì thông tin càng nhiều và vector ngữ cảnh không thể nén được hết thông tin lại, từ đó giảm độ chính xác khi decoder thực hiện sinh chuỗi đầu ra

⇒ Bahdanau Attention ra đời để giải quyết vấn đề này

3. Ý tưởng chính

Thay vì nén toàn bộ thông tin của input vào 1 vector ngữ cảnh duy nhất để decoder sử dụng thì trong Bahdanau Attention, với mỗi bước dịch của decoder, ta sẽ tính lại vector ngữ cảnh để xác định xem nên tập trung vào phần nào của input hay nói cách khác, để tính độ tương quan giữa các từ ở input với từ đang xét ở output

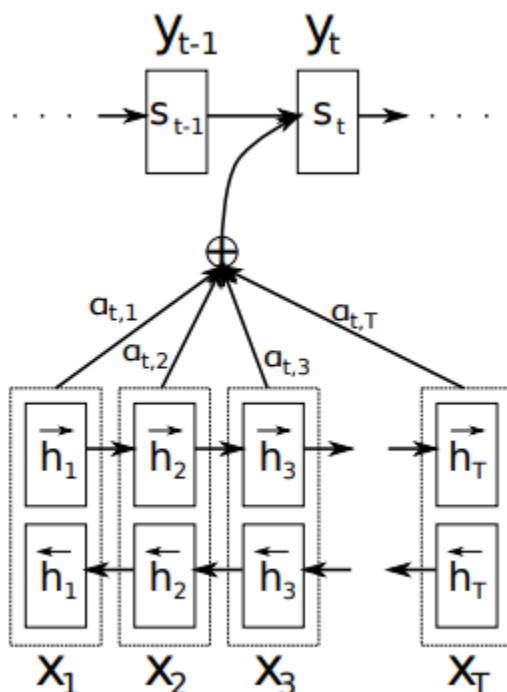
4. Cơ chế hoạt động

Encoder:

Ở phần này cấu trúc được sử dụng là bidirectional RNN để tổng hợp thông tin không chỉ của từ đang xét mà của cả các từ xung quanh nó. BiRNN gồm 2 phần:

- Phần RNN tiến xét các từ theo thứ tự từ x_1 đến x_T với T là độ dài của input, sau đó tính các hidden state tiến tương ứng là $h_1, h_2, \dots, h_T$ (tiền). Các hidden state này chứa thông tin từ đầu câu đến từ hiện tại
- Phần RNN lùi xét các từ theo thứ tự ngược lại, từ x_T đến x_1 , sau đó tính các hidden state lùi tương ứng là $h_T, h_{T-1}, \dots, h_1$ (lùi). Các hidden state này chứa thông tin từ cuối câu đến từ hiện tại

Khi này tại mỗi vị trí i , BiRNN tạo ra 2 hidden state là h_i (tiền) và h_i (lùi), ta tiến hành ghép 2 vector hidden state này lại thành 1 vector hidden state tổng hợp là $h_i = [h_i \text{ (tiền)}; h_i \text{ (lùi)}]$. Các hidden state này sẽ được chuyển cho decoder sử dụng



Decoder:

Tại mỗi bước thứ i , xác suất để decoder sinh ra từ y_i khi biết các từ đã sinh ra trước đó cùng với input là:

$$p(y_i | y_1, \dots, y_{i-1}, \mathbf{x}) = g(y_{i-1}, s_i, c_i)$$

trong đó g là 1 hàm phi tuyến, s_i là hidden state của decoder tại bước thứ i :

$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$

với y_{i-1} là từ được sinh ra ở bước thứ $i - 1$, s_{i-1} là hidden state tại bước thứ $i - 1$, c_i là vector ngữ cảnh được tính tại bước thứ i còn f là 1 hàm phi tuyến. Như đã nói ở trên, vector ngữ cảnh c_i này được tính riêng cho mỗi bước i bằng cách tính tổng có trọng số của các hidden state h_j đã tính ở phần encoder:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

Các trọng số $\alpha_{i,j}$ thể hiện mức độ quan trọng của hidden state h_j với decoder tại bước sinh output thứ i , thực chất là các alignment score $e_{i,j}$ dùng để đo sự liên quan giữa hidden

state thứ $i - 1$ ở output với phần input xung quanh vị trí j , sau khi thông qua hàm softmax để chuẩn hóa về khoảng $[0, 1]$. Các tham số e_{ij} được tính như sau:

$$e_{ij} = a(s_{i-1}, h_j)$$

trong đó a là một mạng nơ-ron truyền thẳng

5. Kết luận

- Bahdanau Attention cho decoder nhìn lại toàn bộ input để tính vector ngữ cảnh
- Vector ngữ cảnh thay đổi theo từng bước decode
- Vẫn còn hạn chế là xử lý tuần tự nên huấn luyện chậm và chưa giải quyết được vanishing gradient
- Đây là nền móng của cơ chế Attention trong Transformer

6. Tham khảo

<https://arxiv.org/pdf/1409.0473>

2.5. Encoder (Phạm Hà Anh)

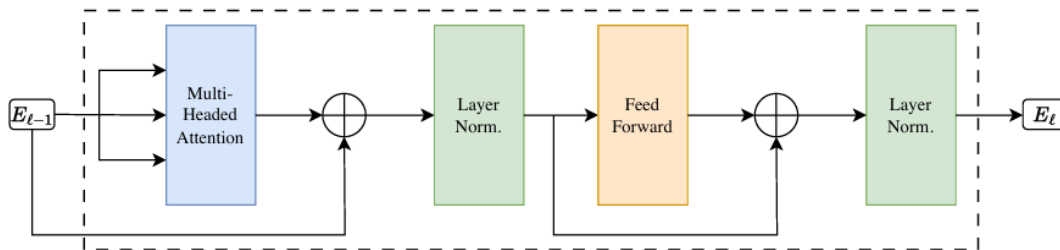


Figure 5: Encoder layer of the original Transformer architecture. Note that the $\oplus$ sign in the diagram corresponds to the skip connections.

Encoder được cấu tạo từ một chuỗi gồm N lớp encoder. Mỗi lớp encoder bao gồm các thành phần sau: Đầu tiên: Input $\rightarrow$ Tokenization $\rightarrow$ Embedding vector ($len \times 512$)

- + Cộng Positional Encoding
- + Lặp 6 lần (mỗi lần là một tầng encoder):
 - a. Multi-Head Self-Attention $\rightarrow$ cộng residual $\rightarrow$ LayerNorm
 - b. Feed-Forward Network $\rightarrow$ cộng residual $\rightarrow$ LayerNorm

Output encoder của tầng 6 $\rightarrow$ decoder

*CHI TIẾT:

Multi-headed self-attention sublayer:

+ Tầng con này bao gồm cơ chế chú ý đa đầu được mô tả chính xác như trong phần trên. Chữ "self" trong tên gọi đề cập đến thực tế rằng cả vector đầu vào là truy vấn

(query) và vector đầu vào là khóa/giá trị (key/value) đều đến từ cùng một đầu vào, nghĩa là ta có $XQ=XK=XV$. Trong Hình trên, điều này tương ứng với việc ba mũi tên đầu vào đều xuất phát từ tầng phía trước.

Layer normalization sublayer:

Thành phần	Mô tả	Kích thước đầu vào (input)	Kích thước đầu ra (output)	Tham số chính
Multi-Headed Attention	Attention đa đầu dạng self-attention: query, key, value đều từ cùng một nguồn (E_{t-1}). Cho phép mỗi token học quan hệ với các token khác.	(L, d_model) với d_model=512	(L,d_model)	- h=8 heads, mỗi head có W^Q , W^K , W^V kích thước (512, 64) - W^O kích thước (512, 512)
⊕ (Skip connection)	Cộng đầu vào của sub-layer (E_{t-1}) với đầu ra của Multi-Headed Attention. Giảm hiện tượng vanishing gradient.	- Nhánh 1: output của Multi-Headed Attention (L,512) - Nhánh 2: E_{t-1} (L,512)	(L, 512) (tổng theo từng phần tử)	Không có tham số

Layer Norm.	Layer Normalization trên output của skip connection. Đưa dữ liệu về phân phối chuẩn (mean 0, var 1) rồi nhân với hệ số tỉ lệ (γ) và cộng bias (β).	(L, 512)	(L, 512)	- γ (gain): (512,) - β (shift): (512,) Cả hai đều là tham số học được
Feed Forward	Mạng truyền thẳng (FFN) gồm 2 lớp tuyến tính với ReLU ở giữa. Áp dụng độc lập từng vị trí.	(L, 512)	(L, 512)	- W1 (512, 2048), b1 (2048) - W2 (2048, 512), b2 (512)
$\oplus$ (Skip connection)	Cộng đầu vào của Feed Forward (Output của Layer Norm bước 3) với đầu ra của Feed Forward.	- Nhánh 1: output của Feed Forward (L,512) - Nhánh 2: output của Layer Norm bước 3 (L,512)	(L, 512)	Không có tham số
Layer Norm.	Layer Normalization lần cuối, chuẩn hóa kết quả sau skip connection. Output chính là E_t – đầu ra của encoder layer này, Sau đó làm input cho encoder layer tiếp theo (hoặc đi đến decoder nếu là tầng cuối cùng).	(L, 512)	(L, 512)	- γ (512,), β (512,) (các tham số khác với Layer Norm ở bước 3)

Lưu ý:

- + E_{t-1} là output của encoder layer trước đó (với tầng đầu tiên, E_0 chính là embedding + positional encoding).
- + E_t là output của encoder layer hiện tại, có cùng kích thước (L, 512).

- + 6 lớp trong Encoder không chia sẻ tham số/trọng số (weights) với nhau. Chúng có cấu trúc giống hệt nhau, nhưng các ma trận trọng số trong mỗi lớp là hoàn toàn độc lập và được học riêng biệt.
- + Xuyên suốt tất cả các lớp, kể cả embedding layer, kích thước vector luôn cố định ở $d_{\text{model}} = 512$ để Residual Connection có thể hoạt động.
- + Tính độc lập của FFN: Khối Feed-Forward áp dụng song song và riêng rẽ cho từng token. FFN không "kết nối" các từ với nhau.
- + Cơ chế Attention trong Encoder là Không che (Unmasked): Từ đứng trước hoàn toàn được "chú ý" tới các từ đứng sau để hiểu trọn vẹn ngữ cảnh ngữ nghĩa

Ví dụ: Input: "Tôi đang training **mạng nơ-ron** bằng một **tập dữ liệu** mới để đánh giá xem **nó** có tốt với **dữ liệu này** ko?".

- Từ "**mạng nơ-ron**" biến thành một dãy 512 số: [0.85, -0.12, 0.43, ..., 0.91]
- Từ "**nó**" biến thành một dãy 512 số: [0.10, 0.02, -0.05, ..., 0.11] (*Lúc này vector của từ "nó" chỉ mang ý nghĩa chung chung của một đại từ*)

Sau đó, mô hình cộng thêm **Positional Encoding** vào các vector này để giữ thứ tự trước sau. Sẽ thu được ma trận đầu vào E_0 đi vào tầng Encoder đầu tiên.

Kết quả tính toán (Attention Score) sẽ trả về tỷ lệ phần trăm "sự chú ý" của từ "**nó**" phân bổ cho các từ trong câu:

- Điểm chú ý đối với chính nó: 5%
- Điểm chú ý đối với từ "**tập dữ liệu**": 1%
- Điểm chú ý đối với từ "**mạng nơ-ron**": **85%**

Khi ra khỏi lớp Encoder 6, tất cả các từ vẫn giữ nguyên kích thước vector ban đầu (512 chiều), nhưng giá trị các con số bên trong đã thay đổi hoàn toàn:

- Vector ban đầu của từ "**nó**": [0.10, 0.02, -0.05, ..., 0.11] (Không có thông tin).
- Vector đầu ra cuối cùng của từ "**nó**": [0.82, -0.45, 0.73, ..., 0.88] (Đã biến đổi mang cấu trúc năng lượng ngữ nghĩa trùng khớp với "**mạng nơ-ron**").

Ma trận context này được chuyển sang cho Decoder để dịch một cách chính xác.

Decoder Layer chi tiết

1. Masked Multi-Head Attention

1. Vấn đề là gì

Ở đây thuật ngữ Multi-Head Attention đã được trình bày cơ chế chi tiết ở lớp Encoder trước đó bây giờ chúng ta đã có thêm 1 từ “Masked” vậy tại sao ta phải “Masked” ?

Hãy nhớ lại nguyên lý cốt lõi của khối Decoder: **Sinh văn bản tự hồi quy (Auto-regressive)**. Nghĩa là nó phải đoán từ thứ t dựa trên các từ từ 1 đến $t-1$.

Chẳng hạn như bạn đang làm một bài thi điền từ vào chỗ trống: “Tôi yêu ____”.

- Để điền được chữ “mèo”, bạn chỉ được phép đọc “Tôi yêu”.
- Nếu hệ thống cho phép từ “yêu” nhìn thấy chữ “mèo” ở tương lai trước khi nó thực sự được sinh ra, thì đó gọi là “nhìn trộm” (data leakage/cheating). Mô hình sẽ học thuộc lòng thay vì thực sự hiểu cách dự đoán.

2. Bản chất Toán học: Sự thanh lịch của vô cực ($-\infty$)

Thuật toán Masked Self-Attention vẫn sử dụng chung nền tảng Scaled Dot-Product Attention, nhưng nó can thiệp vào ngay trước bước tính xác suất (Softmax).

Công thức gốc là:

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$